컴퓨터메이트 개발자 교육 · 반나절 4 시간

Claude Code 로 하는 바이브 코딩

스마트 팩토리 디지털 트윈을 만든 20 개의 프롬프트 — 실습 워크북

항목	내용
성명	
소속	
실습일	
내 업무에서 만들고 싶은 디지털 트윈	

이 워크북은 실습 단계, 기입할 프롬프트 양식, 검증 기록표, 프롬프트 기록 틀을 담고 있습니다. 실습 자료(factory-twin-lab.zip)와 함께 씁니다.

㈜유토닉스

0. 실습 안내

완료 기준

> **완료** 화면 확인 + 테스트·타입 체크·빌드 통과 + 워크북 기록. 세 가지가 모두 채워져야 "완료"입니다.

시간표

시간	구분	내용
13:00–13:45	강의	도입 · I 방법론
13:45–14:15	강의	II 마스터 프롬프트와 설계
14:15–14:25	휴식	
14:25–15:05	강의	III 프롬프트 여정 · IV 자체 검증
15:05–15:15	강의	V 팀 규칙 · 기록 · 실습 안내
15:15–15:25	휴식	
15:25–16:50	실습	0 준비(10) → 1 따라하기(35) → 2 고쳐보기(35) → 공유(5)
16:50–17:00	마무리	실습 3 과제 안내 · 질의응답

실습 시간(분)은 예정입니다. 실제 진행에 따라 강사가 조정합니다.

준비물

☐	항목	확인 메모
☐	Node.js 22 이상 (node -v 로 확인)	
☐	Git	
☐	Claude Code 설치와 로그인	
☐	factory-twin-lab.zip (강사가 나눠 줌)	

실습 0. 준비 (10 분 예정)

```
# zip 을 푼 폴더에서
cd factory-twin-lab
npm ci
npm test
npm run dev        # 터미널에 나온 주소를 브라우저로 연다
claude             # 새 터미널에서 Claude Code 시작
```

실습 자료 구성

위치	내용
LAB.md	실습 안내(준비·과제·완료 기준·막혔을 때)
CLAUDE.md	교육용 팀 규칙

위치	내용
prompts/	프롬프트 기록집 20 건 + 템플릿
lab/01-follow.md · 02-fix.md · 03-record.md	실습 1·2·3 카드
git 태그 step-1 ~ step-5	단계별 완성 지점(막히면 이동)
브랜치 lab/bug-agv · lab/answer-line3	실습 2 버그 상태 · 실습 1 예시 정답

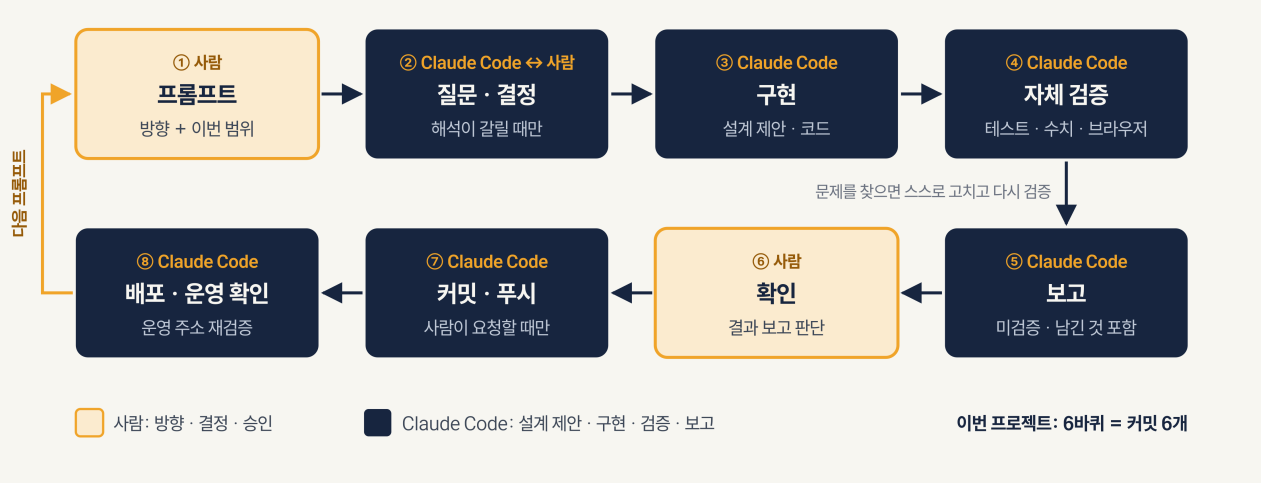
약속 두 가지

> **하나. 완료 게이트** 테스트 → 타입 체크(0 에러) → 빌드 성공 전에는 "완료"라고 하지 않습니다. AI 의 "됐습니다"도 이 기준으로 확인합니다.

> **둘. 기록** 입력한 프롬프트는 원문 그대로, 결과와 검증은 이 워크북에 적습니다. 기록이 다음 프로젝트의 출발점이 됩니다.

방법론 한 장 요약

> **한 문장** 사람은 방향·범위·결정·승인을 맡고, Claude Code 는 설계 제안·구현·검증·보고를 맡는다. 그리고 둘 사이의 약속(완료 기준·커밋 규칙·정직한 보고)은 대화가 아니라 규칙 파일에 둔다.



단계	핵심	프롬프트
0 준비	워크트리 · CLAUDE.md · MCP	—
1 마스터 프롬프트	로드맵 전체 + 이번 범위	P01
2 진행 중 교정	작업 중 끼어들기, 참고 범위 지정	P02
3 위임과 결정 분리	짧게 위임, 결정은 AI 가 묻게	P03, P06

단계	핵심	프롬프트
4 자체 검증 루프	수치·대조·A/B·화면	P01, P03, P11, P16
5 체크포인트 커밋	"커밋하고 main 에 올려 줘"	P04, P05, P07, P09, P15, P17
6 독립 점검	관점별 서브에이전트 리뷰	P08
7 설명 먼저	구조 변경은 설명부터	P10
8 배포·운영 관찰	운영 주소에서 재검증	P12–P14
9 마무리	종료·정리·동기화	P18

1. 따라하기 — 3 라인 추가 (35 분 예정)

카드: lab/01-follow.md. 쓰는 방법론 단계: M1 범위 · M4 검증 · M5 커밋

레이아웃(src/data/factoryLayout.ts)에 생산 라인을 하나 더 추가합니다. 프롬프트 5 요소를 채운 뒤 Claude Code 에 입력합니다.

1 단계. 프롬프트 5 요소 기입

Context	이 저장소가 무엇이고, 무엇을 바꾸려는지
Goal	지금 원하는 결과 한 가지
Constraints	지킬 것(기존 1·2 라인과 테스트를 깨지 않기 등)
Output	받고 싶은 산출물(바뀐 파일 목록과 이유 등)
Verify	AI 가 스스로 확인할 방법(수치·게이트·화면)

예시(그대로 써도 됩니다)

```
Context: 이 저장소는 스마트 팩토리 디지털 트윈이다. 레이아웃은 src/data/factoryLayout.ts 에 있다.
Goal: 생산 라인을 하나 더(3 라인) 추가하고 싶다.
Constraints: 기존 1·2 라인 동작과 테스트를 깨지 말 것. AGV 정류장도 추가할 것.
Output: 바뀐 파일 목록과 이유.
Verify: 2 시간 시뮬레이션으로 라인별 생산량을 비교하고,
        테스트·타입 체크·빌드 결과를 보여 줘.
```

2 단계. 결과 확인

☐	항목	내 결과
☐	화면에 3 라인이 보인다	
☐	공정 흐름 카드에 3 라인 탭이 생겼다	
☐	AI 가 라인별 생산량 비교 수치를 보여 줬다	

☐	항목	내 결과
☐	npm test 통과	
☐	npx tsc --noEmit 0 에러	
☐	npm run build 성공	

3 단계. 라인별 생산량 전/후 기록

라인	추가 전 생산량	추가 후 생산량	비고
1 라인			
2 라인			
3 라인	—		
합계			

시뮬레이션 시간: _____ 시간 / 시드: _____

4 단계. 체크포인트

결과를 확인했으면 "커밋해 줘"로 로컬 커밋을 남깁니다. 정답 예시는 브랜치 lab/answer-line3 에서 비교해 볼 수 있습니다.

커밋 해시	
예시 정답과 다른 점	

2. 고쳐보기 — AGV 하나가 운반을 멈춘다 (35 분 예정)

카드: lab/02-fix.md. 쓰는 방법론 단계: M7 설명 먼저 · M4 수치 검증

1 단계. 버그 브랜치로 이동

```
git switch lab/bug-agv
npm test
npm run dev        # 10 배속으로 AGV 목록을 지켜본다
```

2 단계. 증상 기록

언제(배속·시뮬레이션 시각)	
어느 AGV	
보이는 증상	배터리·운반 횟수·상태 등
다른 AGV 와 다른 점	

3 단계. "설명해 줘" 프롬프트 기입

바로 고치게 하지 말고, 원인과 해결 방안부터 설명받습니다. 아래는 프롬프트 기록집 10 번의 교육용 틀입니다.

```
Context: 점검에서 남긴 과제(<문제>)가 있다.
Goal: 바로 고치지 말고 해결 방안을 설명해 달라.
Output: 원인(코드 위치), 추천안과 대안 비교, 예상 효과(수치), 바뀌는 파일, 검증 계획.
Constraints: 효과를 과장하지 않는다. 결정은 내가 한다.
```

내가 입력한 프롬프트

4 단계. 가설과 측정

가설	확인 방법(무엇을 재나)	결과

측정 항목	수정 전	수정 후
문제 AGV 의 운반 횟수		
문제 AGV 의 최저 배터리		

5 단계. 원인과 수정

원인(코드 위치)	
수정 내용	
추가한 회귀 테스트 이름	
게이트 결과	테스트 · 타입 체크 · 빌드

3. 기록하기 — 내 도메인 디지털 트윈 (과제)

카드: lab/03-record.md. 쓰는 방법론 단계: M1~M5 전체. 완료 기준: 마스터 프롬프트 + 기록 3 건 + 1 단계 화면

1 단계. 도메인 고르기 — 3 기준

☐	항목	내 도메인에서
☐	상태가 계속 바뀌는 대상이 있다(설비·차량·재고·서버 등)	
☐	지금은 한눈에 보기 어렵다	
☐	숫자로 확인할 지표가 있다(처리량·가동률·지연 등)	
고른 도메인	예: 물류센터, 데이터센터 서버룸, 매장	

2 단계. 마스터 프롬프트 기입

전체 로드맵(단계 목록)을 주고, 이번 범위는 좁힙니다.

전체 로드맵	1 단계 … / 2 단계 … / 3 단계 …
Context	
Goal(이번 범위)	
Constraints	
Output	
Verify	

3 단계. AI 의 질문과 나의 답

AI 의 질문 또는 제안	나의 결정	이유

4 단계. 프롬프트 기록 3 건

prompts/00-template.md 와 같은 항목입니다. 원문은 고치지 않고, 고칠 생각은 메모로 따로 적습니다.

기록 1

번호 · 제목	
원문	입력한 프롬프트를 고치지 않고 그대로
상황	직전에 무엇이 끝났고, 왜 이 프롬프트를 썼는지
Claude Code 가 한 일	사용한 기능 포함(질문·서브에이전트·스킬·MCP 등)
사람이 정한 것	AI 의 질문 또는 제안 → 나의 결정
검증	테스트·타입 체크·빌드, 측정 수치, 브라우저 확인
결과(커밋)	
방법론 포인트	이 프롬프트에서 배울 점

기록 2

번호 · 제목	
원문	입력한 프롬프트를 고치지 않고 그대로
상황	직전에 무엇이 끝났고, 왜 이 프롬프트를 썼는지
Claude Code 가 한 일	사용한 기능 포함(질문·서브에이전트·스킬·MCP 등)
사람이 정한 것	AI 의 질문 또는 제안 → 나의 결정
검증	테스트·타입 체크·빌드, 측정 수치, 브라우저 확인
결과(커밋)	
방법론 포인트	이 프롬프트에서 배울 점

기록 3

번호 · 제목	
원문	입력한 프롬프트를 고치지 않고 그대로
상황	직전에 무엇이 끝났고, 왜 이 프롬프트를 썼는지
Claude Code 가 한 일	사용한 기능 포함(질문·서브에이전트·스킬·MCP 등)
사람이 정한 것	AI 의 질문 또는 제안 → 나의 결정
검증	테스트·타입 체크·빌드, 측정 수치, 브라우저 확인
결과(커밋)	
방법론 포인트	이 프롬프트에서 배울 점

부록 A. 오류 대응표

증상	원인	대응
npm ci 에서 Unsupported engine 경고·실패	Node 버전이 낮음	node -v 확인 → Node 22 이상 설치 후 다시 npm ci
npm run dev 후 다른 포트(5174 등)로 열림	5173 포트를 다른 프로그램이 사용 중	터미널에 표시된 주소로 접속, 또는 기존 개발 서버 종료
화면이 비어 있거나 3D 가 안 보임	브라우저 WebGL 미지원·하드웨어 가속 꺼짐	최신 Chrome 에서 하드웨어 가속을 켜고 다시 열기
claude 실행 시 로그인 안내	Claude Code 로그인 전	안내에 따라 로그인 후 다시 실행
파일 수정·명령 실행 권한을 묻는 창	Claude Code 의 권한 확인	무엇을 바꾸는지 읽고 허용. 모르면 "왜 필요한지 설명해 줘"
git switch 시 local changes 오류	커밋하지 않은 변경이 있음	변경을 커밋한 뒤 브랜치 이동
테스트 실패	수정이 기존 동작을 깨뜨림	테스트를 건너뛰지 말고 "실패 원인을 설명해 줘"

부록 B. 자주 쓰는 문장

상황	문장
시작	"전체는 이렇고, 우선 ○○부터 하자"
참고 자료	"○○만 참조하고 △△는 따르지 마"
위임	"○○는 너가 정해서 처리하자"
다음 단계	"원래 프롬프트 N 단계 진행하자"
검증	"몇 시간 돌려서 이상치를 수치로 보여 줘" / "같은 조건으로 A/B 비교해 줘"
구조 변경	"어떻게 해결할건지 설명해 줘"
원인 불명	"재현 조건을 찾고, 못 찾으면 못 찾았다고 말해 줘"
체크포인트	"커밋하고 main 에 올려 줘"
마무리	"모두 종료하고 마무리하자"

검증 문장 (프롬프트 기록집의 Verify 원문)

기록	Verify
01	시뮬레이션을 2 시간 분량 돌려 이상치를 찾고, 테스트·타입 체크·빌드 결과와 브라우저 화면을 확인한 뒤 보고한다.
03	처리기 집계가 시뮬레이터 실제 값과 일치하는지 테스트로 대조하고, 몇 시간 분량을 돌려 알람이 고장보다 먼저 뜨는 비율을 측정한다.
11	같은 입력 조건으로 A/B 비교하고, 테스트·타입 체크·빌드·FPS 를 확인한다.

기록	Verify
14	배포 번들이 로컬 최신 빌드와 같은지, 콘솔 오류·FPS·주요 클릭 동작을 확인한다. 이상이 있으면 재현 조건을 찾고, 못 찾으면 그렇게 보고한다.
16	같은 측정 방법으로 전후를 비교하고, 수정이 다른 시점(시작 직후 등)에 새 문제를 만들지 않는지 확인한다.

부록 C. 명령어 치트시트

명령	하는 일
git status	바뀐 파일 확인(작업 시작 전 항상)
git branch -a	브랜치 목록
git switch lab/bug-agv	실습 2 버그 브랜치로 이동
git tag	단계 태그 목록(step-1 ~ step-5)
git switch --detach step-3	특정 단계 상태로 이동(막혔을 때)
git log --oneline	커밋 이력
git diff	커밋 전 변경 내용 검토

명령	하는 일
npm ci	의존성 설치(잠금 파일 그대로)
npm run dev	개발 서버
npm test	테스트(Vitest)
npx tsc --noEmit	타입 체크(0 에러가 기준)
npm run build	타입 체크 + 프로덕션 빌드

Claude Code	하는 일
claude	저장소 폴더에서 Claude Code 시작
claude -c	직전 대화 이어서 시작
! <명령>	대화창에서 셸 명령을 직접 실행(결과가 대화에 남음). 예: ! git status
Esc	진행 중인 작업 멈추기(방향을 바꿀 때)
/help	명령 목록 보기

부록 D. 완료 게이트 · 보안 체크리스트

☐	항목
☐	테스트 통과
☐	타입 체크 0 에러

☐	항목
☐	빌드 성공
☐	화면에서 직접 확인(콘솔 오류 없음)
☐	커밋 전 git diff 검토
☐	.env·API 키 등 시크릿을 출력·커밋하지 않음
☐	실제 개인정보(이름·연락처 등)를 프롬프트·데이터에 넣지 않음
☐	외부 이미지·영상·글은 저작권 확인 후 사용(출처 표기)
☐	확인하지 못한 것은 "미검증"으로 기록

마무리 약속

다음 주에 Claude Code 로 해 볼 일	
팀에 공유할 프롬프트 기록 1 건	
우리 팀 CLAUDE.md 에 넣을 규칙 1 가지	